

ISLAMIC UNIVERSITY OF TECHNOLOGY

Board Bazar, Gazipur, Dhaka.



Server Programming: SWE 4537
Extra Assignment on System Design

Student ID: 210042112

Fifth Semester
Department of Computer Science and Engineering
BSc. in Software Engineering

March 24, 2025

Contents

1	Given scenario	3
1.1	CineSync – Virtual Movie Theater	3
1.2	Functional Requirements for CineSync–Virtual Movie Theater	3
1.3	Requirement	4
2	My Design	4
2.1	Non-Functional Requirements for CineSync	4
3	Proposed Diagram	7
3.1	Key Elements	8
3.1.1	Client Layer	8
3.1.2	Edge Layer	8
3.1.3	Real-Time Services	8
3.1.4	Media Piepipeline	8
3.1.5	Data Layer	9
3.1.6	Security Layer	9
3.2	Architectural Patterns & Algorithms	9
3.3	System Design Considerations	9
4	Alternative Approaches for Robustness	10
4.1	Zero-Trust Architecture	10
4.2	Homomorphic Encryption for Media Processing	10
4.3	AI-Driven Threat Detection	10
5	References	10

1 Given scenario

1.1 CineSync – Virtual Movie Theater

CineSync is a **virtual movie theater platform** that allows users to **watch movies together in real-time**, synchronizing playback across multiple devices and enabling interactive features like chat, reactions, and remote control.

1.2 Functional Requirements for CineSync–Virtual Movie Theater

1. Real-Time Synchronized Playback

- Ensures all users in a watch party experience the movie at the same time, with minimal latency.
- Leader (host) can pause, play, rewind, or fast-forward for everyone.
- If a user joins late, they are automatically synced to the current timestamp.

2. Interactive Chat & Reactions

- Real-time chat with emoji and GIF reactions.
- Users can send messages without disrupting the movie.
- Time-stamped comments allow users to react to specific moments.

3. Media File Sharing

- Users can upload and share short video clips, memes, or screenshots within the watch party.
- Shared files appear in a dedicated "Media Gallery" for the session.
- File expiration after the session ends to save storage.

4. Server & Channel-Based Party Rooms

- Users can create CineSync Servers for specific groups (e.g., friends, communities).
- Inside each server, users can create watch party rooms with unique settings (private/public, invite-only, etc.).
- Room history (past movies watched, shared media, chat logs) is stored for members.

1.3 Requirement

Identify **five key non-functional requirements with clear justifications**. With a proper diagram, develop a high-level system design incorporating suitable architectures and algorithms.

Explore alternative system design approaches beyond the concepts covered in class. Consider real-world implementations and industry best practices to create a more robust and efficient system.

2 My Design

2.1 Non-Functional Requirements for CineSync

1. **Low Latency (Less than 200ms) for Real-Time Sync** For a smooth experience where people from different locations are watching or interacting with the same content at the same time, there should be very little delay. Actions like playing or pausing should happen instantly to avoid interruptions in the experience.

How to Achieve This:

- **WebRTC Mesh Networking:** This method connects users directly to each other, especially those in the same region, instead of depending on a central server. This helps to reduce delays.
- **Edge Computing:** Services like **AWS Lambda@Edge** or **Cloudflare Workers** process the synchronization information closer to the users, which speeds up the response time.
- **Adaptive Sync Protocol (ASP):** This system adjusts the playback speed automatically based on time checks and connection speeds to ensure smooth synchronization.

How to Measure Success:

- **WebRTC Stats:** Keep track of the delay and any inconsistencies in the connection by checking the round-trip time and the variation in the signal.
 - **Synthetic Monitoring:** Simulate user actions from different parts of the world to test the system's performance in real situations.
2. **99.99% Availability with Multi-Region Redundancy** Watch parties should keep running smoothly even if there's a problem in one region or a big surge in traffic (like during global movie releases).

How to Achieve This:

- **Multi-Region Kubernetes Clusters:** Run services across multiple AWS regions (e.g., US-East, EU-West, AP-South) to avoid downtime.

- **Active-Active Redundancy:** Use **CockroachDB** for a distributed database that switches automatically in case of failure.
- **Chaos Engineering:** Regularly test the system's strength by simulating failures using tools like **Chaos Monkey**.

How to Measure Success:

- **Prometheus + Grafana:** Track system uptime and get alerts if there's a problem.
- **Incident Response Time:** Ensure critical issues are resolved within 15 minutes.

3. **Horizontal Scalability to 1M Concurrent Users** Unexpected traffic spikes (e.g., during holiday events) need to be handled without affecting performance. The system should scale up smoothly to handle more users without any issues.

How to Achieve This:

- **Serverless Orchestration:** Use **AWS Fargate** to manage containerized services and **AWS Lambda** for short-lived tasks, such as chat moderation.
- **Sharded Redis Clusters:** Split session states and chat messages across multiple Redis instances to improve performance and scalability.
- **Kafka for Asynchronous Workloads:** Use Kafka to separate tasks like media transcoding and data analytics by using partitioned topics. This makes it easier to scale these tasks independently.

How to Measure Success:

- **Auto-Scaling Triggers:** Automatically scale up based on the number of WebSocket connections (e.g., over 10,000 per node) or CPU usage (e.g., over 70%).
- **Load Testing:** Use tools like **Locust** or **Gatling** to simulate up to 1 million users and test the system's ability to handle the load.

4. **End-to-End Encryption for Data Privacy** Protect sensitive user data, such as chat logs and shared media, from being accessed or breached by unauthorized parties.

How to Achieve This:

- **AES-256 Encryption:** Use AES-256 to encrypt media files stored in **S3/IPFS** and user data stored in Cassandra, ensuring it remains secure at rest.
- **TLS 1.3 + Mutual TLS (mTLS):** Secure all communications between clients and servers, as well as between internal services, using TLS 1.3 and mutual authentication.
- **Zero-Trust Architecture:** Authenticate devices and users through OAuth 2.0 and use short-lived JWT tokens to ensure that only authorized users can access data.

How to Measure Success:

- **Vulnerability Scans:** Perform monthly security audits using tools like **OWASP ZAP** or **Nessus** to identify potential weaknesses.
- **Key Rotation:** Automatically refresh encryption keys every 90 days to ensure the security of encrypted data.

5. Efficient Media Processing (Transcoding in <5s)

Users expect to be able to instantly watch uploaded videos without any buffering.

How to Achieve This:

- **GPU-Accelerated Transcoding:** Use **NVIDIA T4 GPUs** in Kubernetes pods to process videos quickly in parallel using FFmpeg.
- **Dynamic Quality Adjustment:** Transcode videos into 5 different resolutions (from 144p to 4K) using Per-Title Encoding, a method like Netflix's, to optimize for each video.
- **IPFS Hot/Cold Storage:** Store frequently accessed media in fast, edge-optimized IPFS nodes and move older files to cheaper, slower storage like **S3 Glacier**.

How to Measure Success:

- **Transcoding Speed:** Ensure that 95% of media files are processed in less than 5 seconds, measured using distributed tracing.
- **CDN Cache Hit Ratio:** Aim for a cache hit ratio of over 90% for media served from edge locations.

3 Proposed Diagram

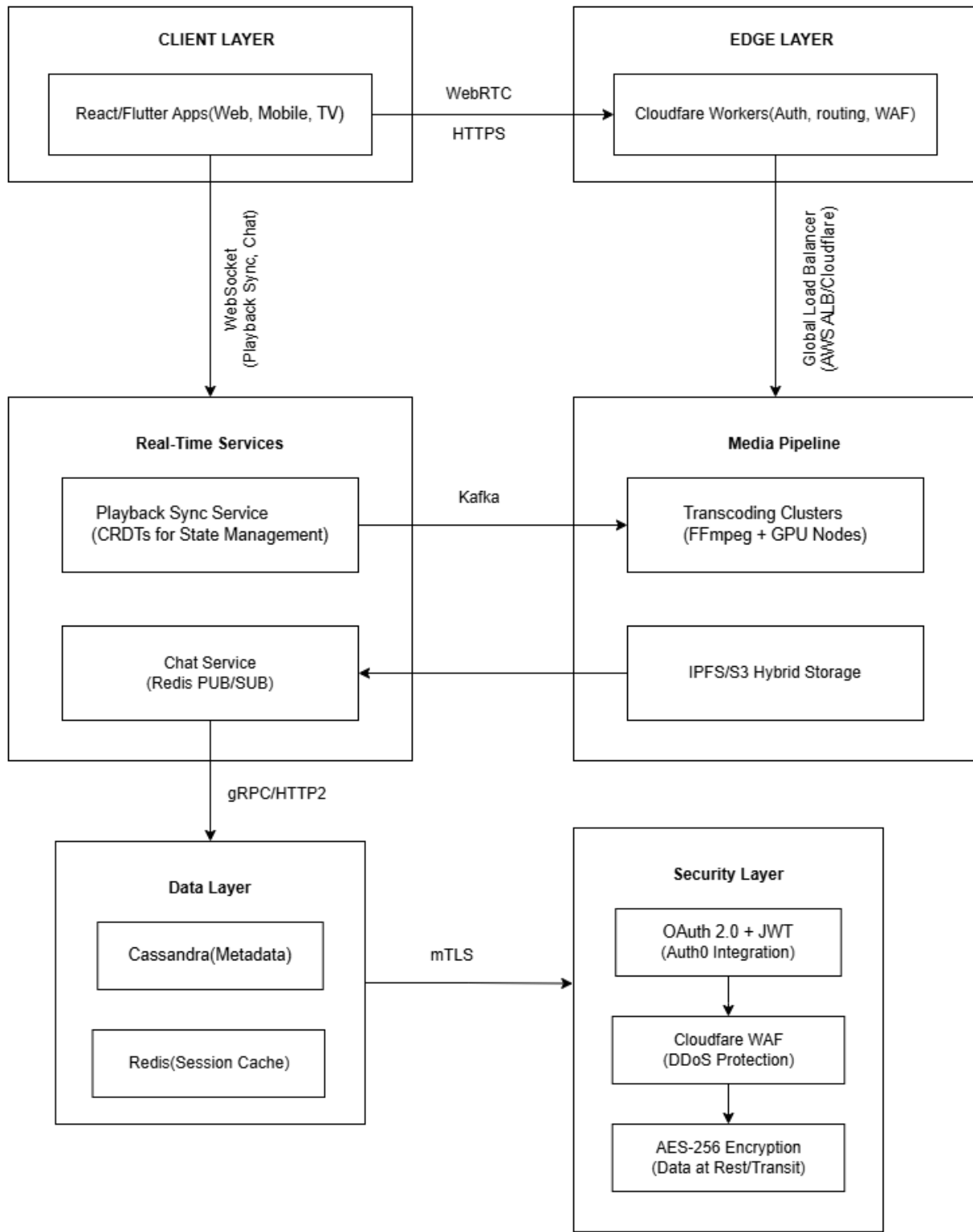


Figure 1: System Architecture Diagram

3.1 Key Elements

3.1.1 Client Layer

- **Architecture:** This part of the system uses a client-server architecture where the client (web, mobile, TV apps) interacts with the server via WebRTC for real-time communication. The applications act as the entry point for users to interact with the system.
- **Algorithm:** WebRTC works on the Peer-to-Peer (P2P) algorithm for direct media transmission, minimizing latency. Data is transferred securely using HTTPS to maintain confidentiality.

3.1.2 Edge Layer

- **Architecture:** This is built on a distributed architecture using Cloudflare Workers to perform request routing, authentication, and security checks closer to the user (at the edge). This reduces the load on central servers and minimizes latency.
- **Algorithm:** The Global Load Balancer uses a latency-based routing algorithm, directing traffic to the nearest edge node to optimize response time. It uses an Algorithmic Load Balancing approach, like Round-Robin or Least Connections.

3.1.3 Real-Time Services

- **Architecture:** This is designed using a microservices architecture where different services like Playback Sync Service and Chat Service operate independently but interact with each other through gRPC/HTTP2 protocols.
- **Algorithm:** The Playback Sync Service employs Conflict-Free Replicated Data Types (CRDTs), an algorithm for distributed state synchronization across devices, ensuring consistency across sessions even with network delays.
- **Chat Service:** The **Redis Pub/Sub** algorithm is used for real-time message delivery, with Kafka employed for asynchronous message processing, analytics, and archiving.

3.1.4 Media Pipeline

- **Architecture:** The media pipeline uses Kubernetes-managed clusters for transcoding, where GPU nodes handle video processing efficiently. This is supported by a hybrid storage architecture combining IPFS (for temporary decentralized storage) and S3 (for permanent backups).
- **Algorithm:** Transcoding uses FFmpeg for video format conversion, and the Hybrid Storage solution uses the Merkle Tree for data deduplication and ensuring consistent access to media files in IPFS.

3.1.5 Data Layer

- **Architecture:** This layer includes a combination of **Cassandra and Redis** to handle both real-time data (session cache, playback state) and persistent metadata (user profiles, video history).
- **Algorithm:** Cassandra uses a Peer-to-Peer Ring topology for distributed database management. The data model optimizes writes and queries on large datasets in real time. Redis, on the other hand, uses in-memory data storage for high-performance caching.

3.1.6 Security Layer

- **Architecture:** This uses a combination of **OAuth 2.0** for identity management, **Cloudflare WAF** for DDoS protection, and **AES-256 encryption** for securing data both at rest and in transit. **Mutual TLS (mTLS)** is used for service-to-service authentication within the system.
- **Algorithm:** OAuth 2.0 is used for secure authorization and authentication, combined with JWTs for managing short-lived tokens. The Cloudflare WAF employs rate-limiting and OWASP security rules to block harmful traffic. For encryption, AES-256 is the standard for symmetric encryption, and key rotation is managed via **AWS KMS** or **HashiCorp Vault** for periodic certificate renewal.

3.2 Architectural Patterns & Algorithms

- **Microservices Architecture:** The system uses independent services for real-time chat, video playback synchronization, and transcoding. This architecture promotes flexibility, scalability, and fault isolation.
- **Event-Driven Architecture:** Using **Kafka** for message archiving and real-time event processing enhances scalability and ensures reliable communication between services.
- **Peer-to-Peer Algorithm:** **WebRTC**'s peer-to-peer communication model significantly reduces latency and offloads traffic, making real-time streaming more efficient.

3.3 System Design Considerations

- **Scalability:** The use of Kubernetes clusters for transcoding, Cassandra for high-write throughput, and Redis for session cache ensures that the system can scale horizontally as more users interact with it.
- **Security:** The combined usage of mTLS, OAuth 2.0 + JWT, Cloudflare WAF, and AES-256 encryption ensures that both data and services are protected, fulfilling modern security best practices.
- **Fault Tolerance:** Microservices like Playback Sync Service and Chat Service ensure that the system can recover from failures in one service without affecting others.

4 Alternative Approaches for Robustness

4.1 Zero-Trust Architecture

- Replace traditional security methods that focus on protecting the network perimeter with continuous checks of users and devices.
- Apply **BeyondCorp** principles to control access based on the user's context, such as their location or device type.

4.2 Homomorphic Encryption for Media Processing

- Encrypt media files in a way that allows video or audio processing (such as transcoding) without needing to decrypt them first.
- Explore libraries like **Microsoft SEAL** for early-stage implementations of this encryption method.

4.3 AI-Driven Threat Detection

- Use machine learning models to spot unusual behavior, such as **strange API** requests or access patterns.
- Implement tools like **AWS GuardDuty** or **Google Chronicle** to analyze and detect threats in real-time.

5 References

1. [webRTC and Mesh network - https://medium.com/@toshvelaga/webrtc-architectures-mesh-mcu-and-sfu-12c502274d7](https://medium.com/@toshvelaga/webrtc-architectures-mesh-mcu-and-sfu-12c502274d7)
2. [Microservices Architecture - https://microservices.io/](https://microservices.io/)
3. [Distributed Database - https://www.geeksforgeeks.org/distributed-database-system/](https://www.geeksforgeeks.org/distributed-database-system/)
4. [Chaos Engineering - https://www.geeksforgeeks.org/what-is-chaos-engineering/](https://www.geeksforgeeks.org/what-is-chaos-engineering/)
5. [Diagram by draw.io](https://draw.io)

The End